

Rust for TouchMotionOS

Complete Learning Guide - tomoro-net & tomoro-store

Generated 2026-07-21

This guide covers every Rust concept, standard-library feature, and third-party crate you need to understand to read, modify, and extend the Rust platform layer of TouchMotionOS. Concepts are ordered from foundational to advanced. Each section includes the concept definition, why it matters for this codebase, and a code example drawn directly from the project.

1. Core Rust Concepts

1.1 Ownership

Every value in Rust has exactly one owner. When the owner goes out of scope, the value is dropped (memory freed). There is no garbage collector. The compiler enforces this at compile time.

- Moving: assigning a value transfers ownership - the original binding is invalid.

```
// move example
let s1 = String::from("hello");
let s2 = s1; // s1 is MOVED into s2
// println!("{s1}"); // compile error: s1 was moved
println!("{s2}"); // OK
```

Note: In tomoro-network, `name: String` inside Interface is owned - cloned when needed via .clone().

1.2 Borrowing & References

Instead of moving, you can lend a value as a reference. References don't own the value. Two kinds: shared (&T) and mutable (&mut T). Rust enforces: either one &mut OR many & at a time - never both.

```
fn read_name(iface: &Interface) -> &str {
    &iface.name // borrow name, no copy
}

// &str is a reference into an existing String
fn wifi_ssid(iface: &str) -> Option<String> { ... }
```

- &str - borrowed string slice (pointer + length). No allocation.

1.3 Lifetimes

References must not outlive the data they point to. The compiler tracks this via lifetimes. Most lifetimes are inferred. You write them explicitly when the compiler cannot figure it out.

```
// explicit lifetime: returned &str lives as long as input &str
fn first_word<'a>(s: &'a str) -> &'a str {
    s.split_whitespace().next().unwrap_or("")
}
```

Note: tomoro-store avoids explicit lifetimes by owning all data (String, Vec, HashMap). You will encounter lifetime errors when trying to store references inside structs - the solution is usually to store owned data instead.

1.4 Structs

Structs group named fields. Methods are defined in impl blocks.

```
// from lib.rs
#[derive(Debug, Clone, Serialize, PartialEq)]
pub struct Interface {
```

```

    pub name: String,
    pub kind: Kind,
    pub up: bool,
    pub carrier: bool,
    pub ipv4: Vec<String>,
    pub ipv6: Vec<String>,
    pub mac: Option<String>,
    pub ssid: Option<String>,
    pub signal_dbm: Option<i32>,
}

```

- pub fields are accessible outside the module.

- de

1.5 Enums with Data

Rust enums can hold data in each variant. This is the core of the store's event system. Unlike C enums, each variant can carry different types and amounts of data.

```

// from store/main.rs - the entire IPC protocol
#[derive(Clone, Debug, Serialize)]
#[serde(tag = "type", rename_all = "snake_case")]
enum Event {
    List { games: Vec<ListGame> },
    Progress { id: String, pct: u8, bytes_done: u64, bytes_total: u64 },
    Installed { id: String },
    Error { id: String, msg: String },
    Launched { id: String },
}

```

serde(tag = "type") means the JSON output includes a "type" field with the variant name. rename_all = "snake_case" converts Installed -> "installed", etc.

1.6 Pattern Matching

match is Rust's primary control flow for enums and complex values. The compiler forces you to handle every case (exhaustiveness).

```

// main.rs dispatch
match args.first().map(String::as_str) {
    None | Some("status") => { println!("{}", ...) }
    Some("watch") => { ... }
    Some("scan") => { ... }
    Some(other) => { eprintln!("unknown: {other}"); }
}

// let-else for early return on None
let Some(iface) = args.get(1) else {
    fail("usage: tomoro-net scan <iface>");
};

```

- | in arms combines patterns (None | Some("status")).

- le

1.7 Option<T>

Rust has no null. Optional values use `Option<T> = Some(T) | None`. The compiler forces you to handle both cases before using the inner value.

```
pub mac: Option<String>,    // may or may not have a MAC address

// common combinators
let name: Option<String> = read_trim(&path); // returns None on IO error
name.as_deref()          // Option<String> -> Option<&str>
    == Some("1")          // compare without unwrapping

// find_map: iterate until Some
lines.find_map(|l| l.trim().strip_prefix("SSID: ").map(str::to_string))
```

1.8 Result<T, E> and Error Handling

Functions that can fail return `Result<T, E> = Ok(T) | Err(E)`. The `?` operator propagates errors up the call stack automatically.

```
// custom error type
enum WifiError {
    Command(String),
    Failed(String),
}

impl std::fmt::Display for WifiError { ... }

fn wpa_cli(iface: &str, args: &[&str]) -> Result<String, WifiError> {
    let out = Command::new("wpa_cli")
        .args(args)
        .output()
        .map_err(|e| WifiError::Command(e.to_string()))?; // ? propagates
    Ok(stdout)
}
```

- `?` desugars to: `if Err(e) { return Err(e.into()); }`

- m

1.9 Closures

Closures are anonymous functions that capture variables from their enclosing scope. Written as `|args| body`.

```
// capture id and tx from outer scope
let cleanup = |e: WifiError| {
    wpa_cli(iface, &["remove_network", id]).ok();
    e // return the error unchanged
};
wpa_cli(iface, &["set_network", id, "ssid", &quoted_ssid]).map_err(cleanup)?;
```

```
// short closure in iterator chain  
nets.sort_by_key(|n| -n.signal_dbm);
```

2. Standard Library - Key Modules

2.1 std::fs - Filesystem

Synchronous filesystem operations. Used in the network crate and in spawn_blocking contexts in the store.

```
use std::fs;

fs::read_to_string("/proc/net/wireless") // -> Result<String>
fs::read_dir("/sys/class/net")           // -> Result<ReadDir>
fs::create_dir_all(&path)                 // create all parent dirs
fs::remove_file(&path)                   // delete file
```

2.2 std::path::{Path, PathBuf}

Path is borrowed (like &str), PathBuf is owned (like String). Use .join() to append segments. Use .parent() to get directory.

```
fn manifest_path() -> PathBuf {
    PathBuf::from("/var/lib/tomoro")
        .join("manifest.json")
}

let game_dir = games_dir().join(&id); // /var/lib/tomoro/games/tomoro-breakout
if let Some(parent) = path.parent() {
    fs::create_dir_all(parent).await?;
}
```

2.3 std::process::Command

Spawns child processes. Used to call wpa_cli and iw in the network crate.

```
use std::process::Command;

let out = Command::new("wpa_cli")
    .arg("-p").arg("/run/wpa_supplicant/control")
    .arg("-i").arg(iface)
    .args(args)           // &[&str] slice
    .output()?;           // waits for completion, captures stdout+stderr

out.status.success()     // bool - exit code 0?
out.stdout               // Vec<u8>
String::from_utf8_lossy(&out.stdout).trim().to_string()
```

2.4 std::collections::HashMap

Key-value map. Used for the game manifest (game_id -> ManifestEntry).

```
use std::collections::HashMap;

type Manifest = HashMap<String, ManifestEntry>;

let mut manifest: Manifest = HashMap::new();
manifest.insert(id.clone(), ManifestEntry { ... });
```

```
let entry = manifest.get(id) // -> Option<&ManifestEntry>
    .ok_or_else(|| format!("game '{id}' not installed"))?;
```

2.5 std::env

Read environment variables at runtime. Used for all config in the store daemon.

```
std::env::var("TOMORO SOCK")
    .unwrap_or_else(|_| "/run/tomoro/store.sock".to_string())

std::env::args()           // iterator over CLI arguments
    .skip(1)               // skip argv[0] (program name)
    .collect::<Vec<String>>()
```

2.6 Iterators

Rust iterators are lazy - no work until consumed. Chaining adapters builds a pipeline that executes in one pass. Critical to read fluently.

```
// find: first match
out.iter_mut().find(|(n, _)| n == &name)

// find_map: first Some()
lines.find_map(|l| l.trim().strip_prefix("SSID: ").map(str::to_string))

// map + collect
manifest.iter().map(|(id, e)| ListGame {
    id: id.clone(), version: e.version.clone(), installed: true
}).collect::<Vec<_>>()

// sort
out.sort_by(|a, b| a.name.cmp(&b.name));
nets.sort_by_key(|n| -n.signal_dbm);

// filter + flatten
entries.flatten() // Iterator<Item=Result<T>> -> skip Err

// nth: skip n items, take next
parts.nth(2)? // field index 2 in whitespace-split line
```

2.7 String Manipulation

```
s.trim()                // strip whitespace
s.trim_end_matches(':') // strip specific char from end
s.split_whitespace()    // split on any whitespace
s.split('\t')           // split on tab
s.lines()               // line iterator
s.starts_with("FAIL")   // prefix check
s.contains("WPA")       // substring check
s.strip_prefix("SSID: ") // -> Option<&str>
format!("{}", ssid)     // string formatting
s.parse::<f64>()        // -> Result<f64, ParseFloatError>
```

2.8 Unsafe Rust & FFI (network crate)

The `if_addresses()` function calls the POSIX `getifaddrs(3)` C function directly. This requires `unsafe` because Rust cannot verify raw pointer safety.

```
use std::ffi::CStr;

unsafe {
    let mut ifap: *mut libc::ifaddrs = std::ptr::null_mut();
    if libc::getifaddrs(&mut ifap) != 0 { return out; }

    let mut cur = ifap;
    while !cur.is_null() {
        let ifa = &*cur;          // dereference raw pointer -> reference
        cur = ifa.ifa_next;
        let name = CStr::from_ptr(ifa.ifa_name) // null-terminated C string
            .to_string_lossy().into_owned();
        // cast pointer to concrete sockaddr type
        let sa = &*(ifa.ifa_addr as *const libc::sockaddr_in);
        let ip = std::net::Ipv4Addr::from(u32::from_be(sa.sin_addr.s_addr));
    }
    libc::freeifaddrs(ifap); // must free manually
}
```

- `*mut T` - raw mutable pointer, no borrow-checker protection.

- `*C`

Note: You do not need to write unsafe code - but you must be able to read it to understand `if_addresses()`.

3. Async Rust with Tokio

Async Rust allows many concurrent tasks on a small thread pool without blocking threads. Instead of blocking (sleep, read, write), async functions yield control back to the executor while waiting, letting other tasks run. tokio is the async runtime used in tomoro-store.

3.1 async fn and .await

Any function marked async fn returns a Future. The future does nothing until you .await it. await suspends the current task until the future completes, without blocking the thread.

```
// async fn returns Future<Output = Manifest>
async fn read_manifest() -> Manifest {
    let Ok(text) = fs::read_to_string(&path).await else {
        return HashMap::new();
    };
    // .await yields until file is read
    serde_json::from_str(&text).unwrap_or_default()
}

// calling it
let manifest = read_manifest().await;
```

3.2 #[tokio::main] - The Runtime

tokio::main is a macro that creates the Tokio async runtime and runs your async main function inside it. Without a runtime, futures do nothing - they are inert values.

```
#[tokio::main]
async fn main() {
    let listener = UnixListener::bind(&sock).expect("bind Unix socket");
    loop {
        match listener.accept().await { ... }
    }
}
```

3.3 tokio::spawn - Concurrent Tasks

tokio::spawn runs a future concurrently with the current task. It returns a JoinHandle (ignored here - fire and forget). Each connected client gets its own spawned task.

```
tokio::spawn(handle_client(
    stream,
    tx.clone(),
    http.clone(),
    backend.clone(),
    cid,
));

// inside handle_client: spawn the reader task separately
tokio::spawn(async move {
    while let Ok(Some(line)) = lines.next_line().await {
        // handle commands
    }
});
```

```
    }
  });
```

Note: `async move`: the closure captures variables by move (takes ownership) into the `async` block.

3.4 `tokio::net::UnixListener` / `UnixStream`

Async Unix domain socket server. Flutter connects here; the daemon listens. Each accepted connection is a `UnixStream`.

```
use tokio::net::{UnixListener, UnixStream};

let listener = UnixListener::bind("/run/tomoro/store.sock");
let (stream, _addr) = listener.accept().await; // blocks until connection

// split into independent halves
let (read_half, mut write_half) = stream.into_split();
```

3.5 `into_split()` - Splitting Read/Write

`into_split()` splits one `UnixStream` into an owned read half and write half. This is critical: the reader runs in a spawned task while the writer stays in the outer loop. You cannot have two mutable references to the same stream simultaneously.

```
let (read_half, mut write_half) = stream.into_split();

// reader runs in separate spawned task
let mut lines = BufReader::new(read_half).lines();
tokio::spawn(async move {
    while let Ok(Some(line)) = lines.next_line().await { ... }
});

// writer stays here, forwards events back to Flutter
write_half.write_all(line.as_bytes()).await;
```

3.6 `tokio::io - Async Buffered I/O`

```
use tokio::io::{AsyncBufReadExt, AsyncWriteExt, BufReader};

let mut lines = BufReader::new(read_half).lines();
while let Ok(Some(line)) = lines.next_line().await {
    // process one newline-delimited JSON command
}

write_half.write_all(b"hello\n").await;
```

- `AsyncBufReadExt` adds `.lines()` to any async reader.

- AS

3.7 `tokio::fs - Async Filesystem`

Drop-in async version of `std::fs`. Use inside async functions to avoid blocking the executor.

```
use tokio::fs;
```

```
let text = fs::read_to_string(&path).await?;
fs::create_dir_all(&dir).await?;
fs::write(&path, content).await?;

let mut file = tokio::fs::File::create(&part_path).await?;
file.write_all(&chunk).await?;
```

3.8 tokio::task::spawn_blocking

Blocking CPU work (like tar extraction) would starve the async executor if run directly. `spawn_blocking` offloads it to a dedicated thread pool, returning a future you can `.await`.

```
let extract_result = tokio::task::spawn_blocking(move || {
    // This runs on a blocking thread, not the async executor
    let file = std::fs::File::open(&part_path)?;
    let gz = GzDecoder::new(file);
    let mut archive = tar::Archive::new(gz);
    archive.unpack(&game_dir)?;
    std::fs::remove_file(&part_path)?;
    Ok::(<_, std::io::Error>())
}).await; // wait for the blocking task to finish

match extract_result {
    Ok(Ok(())) => {} // task ran, extraction succeeded
    Ok(Err(e)) => { ... } // task ran, extraction failed
    Err(e) => { ... } // task panicked
}
```

Note: `spawn_blocking` returns `JoinHandle<T>`. `.await` gives `Result<T, JoinError>`. The inner `T` is your return value, hence `Ok(Ok(()))`.

3.9 tokio::process::Command

Async version of `std::process::Command`. Spawn child processes without blocking the executor.

```
use tokio::process::Command;

// launch a game without waiting for it to exit
tokio::process::Command::new("steam-run")
    .arg(&binary)
    .spawn() // non-blocking - returns immediately
    .map_err(|e| format!("spawn failed: {e}"))?;
```

4. Channels & Concurrency

4.1 tokio::sync::broadcast - The Core IPC Pattern

broadcast is a multi-producer multi-consumer channel where every receiver gets every message. In tomoro-store, one channel fans out events to all connected Flutter clients.

```
use tokio::sync::broadcast;

// Create channel with buffer of 256 messages
let (tx, _) = broadcast::channel::<(Option<String>, Event)>(256);

// Each spawned connection handler gets its own receiver
let mut rx = tx.subscribe();

// Send from any task (tx is Clone)
let _ = tx.send((Some(client_id.clone()), Event::Installed { id: id.clone() }));

// Receive in the write loop
loop {
    match rx.recv().await {
        Ok((target, event)) => { /* forward to client */ }
        Err(broadcast::error::RecvError::Lagged(_)) => continue, // missed msgs
        Err(broadcast::error::RecvError::Closed) => break,
    }
}
```

- tx is Clone - share it by cloning into each spawned task.

- tx

4.2 Arc<Mutex<T>> - Shared Mutable State

Arc (Atomically Reference Counted) lets multiple tasks share ownership. Mutex provides exclusive access - only one task holds the lock at a time. The store imports both but the broadcast channel largely replaces the need for shared state.

```
use std::sync::Arc;
use tokio::sync::Mutex; // async-aware mutex (doesn't block the thread)

let state = Arc::new(Mutex::new(HashMap::<String, Status>::new()));

// Clone Arc to share with a spawned task
let state_clone = Arc::clone(&state);
tokio::spawn(async move {
    let mut guard = state_clone.lock().await; // acquire lock
    guard.insert("key".into(), value);
    // lock released when guard drops
});
```

Note: Use `tokio::sync::Mutex` in async code, not `std::sync::Mutex`. Holding a `std::Mutex` across `.await` points can deadlock.

5. Serde - Serialization

5.1 derive(Serialize, Deserialize)

Serde generates JSON serialization/deserialization code at compile time via derive macros. Add `#[derive(Serialize)]` to any struct or enum to enable `.to_string()` via `serde_json`.

```
use serde::{Deserialize, Serialize};

#[derive(Debug, Clone, Serialize, PartialEq)]
pub struct WifiNetwork {
    pub ssid: String,
    pub bssid: String,
    pub signal_dbm: i32,
    pub secured: bool,
}

#[derive(Deserialize)]
struct GameMeta {
    id: String,
    version: String,
    download_url: String,
    binary: String,
    size_bytes: u64,
}
```

5.2 serde Attributes

```
// Enum: add "type" field with snake_case variant name
#[serde(tag = "type", rename_all = "snake_case")]
// -> {"type": "progress", "id": ..., "pct": ...}

// Struct: rename fields in JSON output
#[serde(rename_all = "lowercase")]
// Kind::Ethernet -> "ethernet"

// Skip field if None
#[serde(skip_serializing_if = "Option::is_none")]

// Custom field name
#[serde(rename = "signal_level")]
pub signal_dbm: i32,
```

5.3 serde_json - JSON at Runtime

```
use serde_json::{json, Value};

// Serialize struct to JSON string
serde_json::to_string(&status_struct)?;
serde_json::to_string_pretty(&manifest)?;

// Parse JSON string into typed struct
```

```
serde_json::from_str::<GameMeta>(&text)?;

// Parse into untyped Value
let val: Value = serde_json::from_str(&line)?;
val.get("cmd").and_then(Value::as_str)  // -> Option<&str>

// json! macro - build JSON inline
serde_json::json!({ "error": msg.to_string() })
serde_json::json!({ "connected": ssid })
```

6. Key Third-Party Crates

6.1 request - HTTP Client

Async HTTP client. Used to fetch game metadata from backend and stream download bytes.

```
use request::Client;
use futures_util::StreamExt;

let client = request::Client::new();

// Fetch JSON and deserialize
let meta: GameMeta = client.get(&url).send().await?.json().await?;

// Stream bytes (large download)
let resp = client.get(&meta.download_url).send().await?;
let mut stream = resp.bytes_stream();
while let Some(chunk) = stream.next().await {
    let chunk: bytes::Bytes = chunk?;
    file.write_all(&chunk).await?;
    downloaded += chunk.len() as u64;
}
```

- Cargo.toml: request = { features = ["stream"] } - enables bytes_stream().

- C

6.2 flate2 + tar - Archive Extraction

Decompress .gz and extract .tar in a spawn_blocking closure.

```
use flate2::read::GzDecoder;
use tar::Archive;

let file = std::fs::File::open(&part_path)?;
let gz = GzDecoder::new(file); // wraps file in gzip decoder
let mut archive = tar::Archive::new(gz); // wraps gz in tar reader
archive.unpack(&game_dir)?; // extract all files to game_dir
std::fs::remove_file(&part_path)?; // delete .part file
```

6.3 libc - C Bindings

Provides raw C types and function signatures for syscalls not exposed by std.

```
libc::ifaddrs // C struct
libc::sockaddr_in // IPv4 socket address struct
libc::sockaddr_in6 // IPv6 socket address struct
libc::AF_INET // address family constant
libc::getifaddrs() // C function: enumerate network interfaces
libc::freeifaddrs() // C function: free the linked list
```


7. Cargo & Project Structure

7.1 Workspace Layout

```
# platform/Cargo.toml
[workspace]
resolver = "2"
members = ["crates/network", "crates/store"]

# Each crate has its own Cargo.toml
platform/crates/network/Cargo.toml
platform/crates/network/src/main.rs    # binary entry point
platform/crates/network/src/lib.rs     # library (reusable functions)

platform/crates/store/Cargo.toml
platform/crates/store/src/main.rs      # binary entry point (no lib)
```

- cargo build --release in platform/ builds all workspace members.

- W

7.2 Cargo.toml - Dependencies

```
# platform/crates/store/Cargo.toml
[package]
name = "tomoro-store"
version = "0.1.0"
edition = "2021"

[[bin]]
name = "tomoro-store"
path = "src/main.rs"

[dependencies]
tokio = { version = "1", features = ["full"] }
serde = { version = "1", features = ["derive"] }
serde_json = "1"
reqwest = { version = "0.12", features = ["stream"] }
tar = "0.4"
flate2 = "1"
futures-util = "0.3"
bytes = "1"
```

7.3 Common cargo Commands

```
cargo build          # debug build
cargo build --release # optimized build
cargo check          # type-check without linking (fast)
cargo clippy         # linter - catches common mistakes
cargo test           # run tests
cargo run -- status  # run binary with args
cargo doc --open     # generate and open docs
```

8. Architecture Deep Dive - How It All Fits

8.1 The Protocol (Newline-Delimited JSON)

Flutter and the Rust daemon communicate over a Unix socket using one JSON object per line. This is the entire IPC contract.

```
# Flutter -> daemon (commands)
{"cmd": "list"}
{"cmd": "install", "id": "tomoro-breakout"}
{"cmd": "launch", "id": "tomoro-breakout"}

# daemon -> Flutter (events, streamed continuously)
{"type": "list", "games": [{"id": "...", "installed": true}]}
{"type": "progress", "id": "tomoro-breakout", "pct": 42, ...}
{"type": "installed", "id": "tomoro-breakout"}
{"type": "error", "id": "tomoro-breakout", "msg": "..."}

```

8.2 Client Routing with Option<String>

The broadcast channel carries (Option<String>, Event). The Option<String> is a client ID. None = send to all clients. Some(id) = send only to that client. Each connection handler checks the target before forwarding.

```
// in the write loop
match rx.recv().await {
  Ok((target, event)) => {
    let send = match &target {
      None => true,           // broadcast
      Some(t) => t == &cid,   // only my client
    };
    if send {
      let mut line = serde_json::to_string(&event).unwrap();
      line.push('\n');
      write_half.write_all(line.as_bytes()).await?;
    }
  }
}

```

8.3 Download Pipeline

Full flow for one install command:

- 1. Flutter sends {"cmd": "install", "id": "tomoro-breakout"} over socket.

- 2.

8.4 tomoro-net Watch Mode

Polling loop that only emits when state changes - efficient diffing without inotify.

```
let mut last: Option<Status> = None;
loop {
    let now = tomoro_network::status();
    if last.as_ref() != Some(&now) { // PartialEq compare
        println!("{}", serde_json::to_string(&now).unwrap());
        last = Some(now);
    }
    thread::sleep(Duration::from_secs(interval));
}
```

Note: PartialEq is derived on Status and Interface - allows == comparison of entire structs including nested Vec<String>.

9. Traits You Must Know

9.1 Display and Debug

```
// Debug: auto-derived, used in {:?} format strings
#[derive(Debug)]
struct Foo { x: i32 }
println!("{:?}", foo);    // Foo { x: 42 }

// Display: implemented manually, used in {} format strings
impl std::fmt::Display for WifiError {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        match self {
            WifiError::Command(e) => write!(f, "wpa_cli unavailable: {e}"),
            WifiError::Failed(e) => write!(f, "{e}"),
        }
    }
}
```

9.2 Clone and Copy

```
// Clone: explicit duplication via .clone()
#[derive(Clone)]
struct Interface { name: String, ... } // String requires Clone

// Copy: implicit bitwise copy (no .clone() needed)
// Only for stack types: bool, i32, u8, f64, etc.
#[derive(Copy, Clone)]
pub enum Kind { Ethernet, Wifi } // simple enum, no heap data -> Copy
```

9.3 PartialEq

```
// Enables == and != comparison
#[derive(PartialEq)]
pub struct Status { pub interfaces: Vec<Interface>, pub online: bool }

// Used in watch mode:
if last.as_ref() != Some(&now) { ... }
```

9.4 From / Into / map_err

```
// map_err: convert error type in a Result chain
Command::new("wpa_cli").output()
    .map_err(|e| WifiError::Command(e.to_string()))?;

// e.to_string() calls Display::fmt
// WifiError::Command(e.to_string()) wraps the string in the enum variant
```

10. Concepts Checklist

Use this as a self-assessment. For each concept, can you read the relevant code in the project and explain what it does?

Core Language

- Ownership: who owns a String when it's passed to a function?

- B

Standard Library

- `std::fs`: `read_dir`, `read_to_string`, `create_dir_all`, `remove_file`.

- st

Async / Tokio

- `async fn` / `.await`: what a Future is, why you need `.await`.

- #[

Channels & Concurrency

- `broadcast::channel`: how tx/rx relate, cloning tx, subscribing rx.

- R

Crates

- `request`: `Client::new()`, `.get().send().await`, `.json().await`, `.bytes_stream()`.

11. Recommended Learning Order

If starting from scratch, work through these in order:

1. The Rust Book chapters 1-10

<https://doc.rust-lang.org/book/>

Ownership, borrowing, structs, enums, pattern matching, error handling, traits, generics.

2. The Rust Book chapter 13

<https://doc.rust-lang.org/book/ch13-00-functional-features.html>

Closures and iterators - critical for reading the network crate.

3. Rustlings exercises

<https://github.com/rust-lang/rustlings>

Hands-on exercises for everything in chapters 1-13.

4. Read tomoro-net lib.rs top to bottom

`platform/crates/network/src/lib.rs`

Apply ownership/iterators/Option/Result in a real, small codebase.

5. Tokio tutorial

<https://tokio.rs/tokio/tutorial>

Async basics, spawning tasks, channels. Do the mini-redis project.

6. Read tomoro-store main.rs

`platform/crates/store/src/main.rs`

Apply everything: async, broadcast, reqwest, serde, spawn_blocking.

7. Rustonomicon chapter on FFI

<https://doc.rust-lang.org/nomicon/ffi.html>

Understand the unsafe getifaddrs block.